



Linux Device Driver Development Cont. II

ECE 391 – Computer Systems Engineering

Dong Kai Wang

Department of Electrical and Computer Engineering
University of Illinois at Urbana-Champaign

- **Additional Driver Operations**

- Control operations: `ioctl()`.
- Sleeping.

- **Final Exam Info**

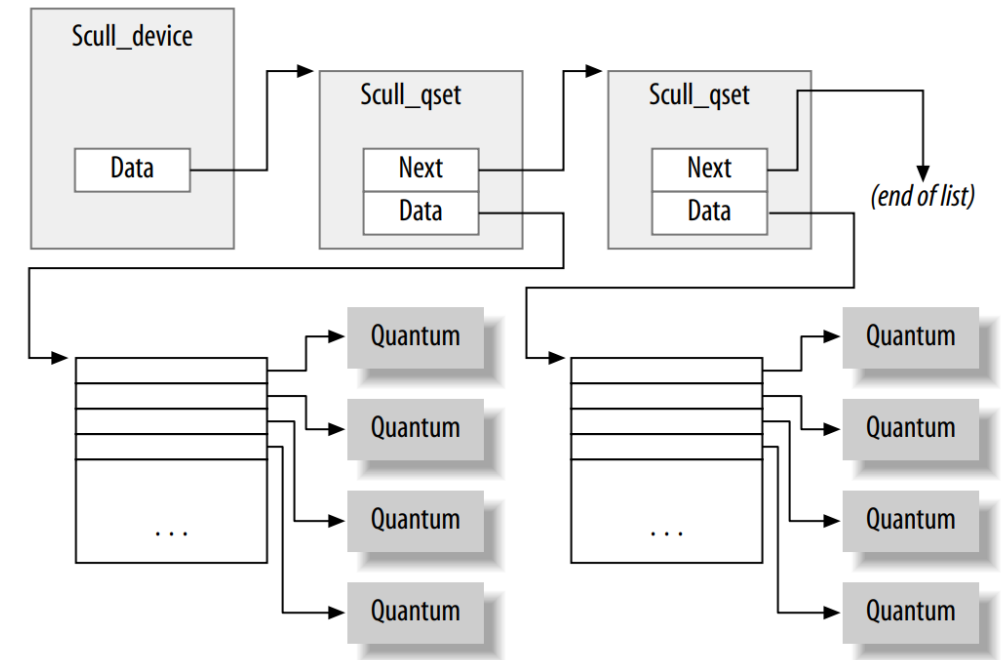
- MP3 Review with TAs.

- *Reference: Linux Device Drivers, 3rd Edition, O'Reilly Media.*

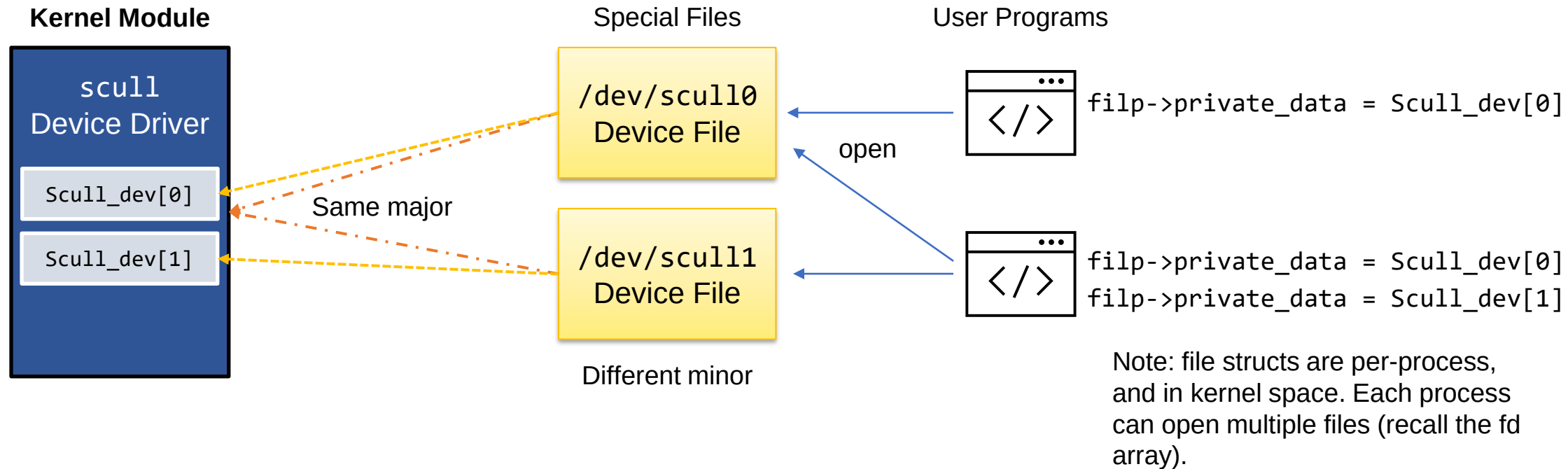
Recap: The scull Device



- **Simple Character Utility for Loading Localities.**
 - A simple character device to store some data internally.
 - *Memory-only* (does not act on actual hardware).
 - Data structures: a linked list of pointer arrays.
- Modified scull driver on course site:
 - Should work on modern Linux kernels.
sculldriver.tar.xz (Try on your own VM)
- In scull each _____ has a `scull_dev` struct storing the linked list of data.
 - A. Driver (**Major**)
 - B. Device (**Minor**)
 - C. Process



- In summary: `scull` maintains an array of struct containing data private to each device (minor number).



- Different devices (minor) sharing the same driver (major) can have different `read()` and `write()` functions.
 - **TRUE**
 - **FALSE**
- Each device file (like `/dev/scull0`) is associated with one `file` struct.
 - **TRUE**
 - **FALSE**
- Device files (like `/dev/scull0`) contain the driver code.
 - **TRUE**
 - **FALSE**

Recall that every process has a file struct * array (*fd*) in kernel that stores information about its **open** files, this is specific to the process. (Note: not the same as *libc* FILE* pointer)

```
struct file_operations *f_op;
```

- File operations associated with file.
- Set initially by kernel on open, can be substituted with other functions in your driver.

```
loff_t f_pos;
```

- Current read/write position within a file.
- Should be directly updated by `llseek` and indirectly updated by read/write file operations.

```
struct dentry *f_dentry;
```

- Stores directory entry structure associated with file. Often not used except for accessing inode (`filp->f_dentry->d_inode`), containing device file's major/minor numbers (`i_major(struct inode* inode)`).

```
void *private_data;
```

- Set to NULL by kernel on open (not always), you can have this pointer point to any data in your driver.

- We may want different file operation functions to handle different types of scull.

```
if (type > SCULL_MAX_TYPE) return -ENODEV;
filp->f_op = scull_fop_array[type];
return filp->f_op->open(inode, filp); /* dispatch to specific open */
```

Set file operations based on device type (**minor** number).
Then call the open function based on device type.

- Next, set the private data pointer for the device.

- Each minor number has its own scull_dev struct.
- Thus, data in linked list is not shared between different device files.

```
/* type 0, check the device number (unless private_data valid) */
dev = (Scull_Dev *)filp->private_data;
if (!dev) {
    if (num >= scull_nr_devs) return -ENODEV;
    dev = &scull_devices[num]; Array of Scull_dev, each containing their own data.
    filp->private_data = dev; /* for other methods */
}
```

```
struct file_operations *scull_fop_array[]={
    &scull_fops, /* type 0 */
    &scull_priv_fops, /* type 1 */
    &scull_pipe_fops, /* type 2 */
    &scull_sngl_fops, /* type 3 */
    &scull_user_fops, /* type 4 */
    &scull_wusr_fops /* type 5 */
};
#define SCULL_MAX_TYPE 5
```

Array of file operation structs, each struct containing function pointers to its own read(), write(), open(), etc.

```
int (*ioctl) (struct inode *inode, struct file *filp, unsigned int
cmd, unsigned long arg);
```

- Allows user space programs to send (usually control) commands to driver.
 - You define the format of arguments, command no. (*there are general guidelines in Linux*).
- Control without ioctl: *why not issue commands through write()*?

```
device_write(...) {
    data = copy_from_user(...);
    if (data == "reset device") { ... }
    else if (data == "change configuration") { ... }
    else // do normal write
}
```

- In many cases, this is fine and sufficient (*escape sequences*), but **what are the drawbacks?**


```
int (*ioctl) (struct inode *inode, struct file *filp, unsigned int  
cmd, unsigned long arg);
```

cmd: you can define command numbers but there are general guidelines in Linux.

- Use a combination of bits based on driver type (choose a magic number, consult ioctl-number.txt), command number, read/write direction, data size.
 - Check `<linux/ioctl.h>`.
- ioctl function usually implemented as a long switch statement.

arg: the second argument is converted to unsigned long but can represent anything depending on the command (you specify the API for users).

- Integer (or any data that fits within long).
- Pointer (to something in user space, will need `copy_from_user` or `_to_user`)
- Unused (or irrelevant to command).

ioctl in the scull Driver



```
switch(cmd) {

#ifdef SCULL_DEBUG
    case SCULL_IOCHARDRESET:
        /*
         * reset the counter to 1, to allow unloading in case
         * of problems. Use 1, not 0, because the invoking
         * process has the device open.
         */
        while (MOD_IN_USE)
            MOD_DEC_USE_COUNT;
        MOD_INC_USE_COUNT;
        /* don't break: fall through and reset things */
#endif /* SCULL_DEBUG */

    case SCULL_IOCRESET:
        scull_quantum = SCULL_QUANTUM;
        scull_qset = SCULL_QSET;
        break;

    case SCULL_IOCSQUANTUM: /* Set: arg points to the value */
        if (!capable(CAP_SYS_ADMIN))
            return -EPERM;
        ret = __get_user(scull_quantum, (int *)arg);
        break;

    case SCULL_IOCTLQUANTUM: /* Tell: arg is the value */
        if (!capable(CAP_SYS_ADMIN))
            return -EPERM;
        scull_quantum = arg;
        break;
```

```
    case SCULL_IOCQQUANTUM: /* Query: return it (it's positive) */
        return scull_quantum;

    case SCULL_IOCXQUANTUM: /* eXchange: use arg as pointer */
        if (!capable(CAP_SYS_ADMIN))
            return -EPERM;
        tmp = scull_quantum;
        ret = __get_user(scull_quantum, (int *)arg);
        if (ret == 0)
            ret = __put_user(tmp, (int *)arg);
        break;

    case SCULL_IOCHQUANTUM: /* sHift: like Tell + Query */
        if (!capable(CAP_SYS_ADMIN))
            return -EPERM;
        tmp = scull_quantum;
        scull_quantum = arg;
        return tmp;

    default: /* redundant, as cmd was checked against MAXNR */
        return -ENOTTY;
}
return ret;
```

- What if an operation like read cannot immediately be completed (e.g. hardware busy)?
- Put the process to sleep until ready (some condition becomes true).

- In Linux, we can use a wait queue: a list of processes waiting for a specific event.

```
DECLARE_WAIT_QUEUE_HEAD(name);
```

- The `wait_event` macro allows us to the process into (interruptible) sleep with some provided condition.

```
wait_event(queue, condition)
wait_event_interruptible(queue, condition)
wait_event_timeout(queue, condition, timeout)
wait_event_interruptible_timeout(queue, condition, timeout)
```

- Another process should perform the wake-up using the `wake_up` macro.

Sleeping Example



```
static DECLARE_WAIT_QUEUE_HEAD(wq);
static int flag = 0;

ssize_t sleepy_read (struct file *filp, char __user *buf, size_t count, loff_t *pos)
{
    printk(KERN_DEBUG "process %i (%s) going to sleep\n",
           current->pid, current->comm);
    wait_event_interruptible(wq, flag != 0);
    flag = 0;
    printk(KERN_DEBUG "awoken %i (%s)\n", current->pid, current->comm);
    return 0; /* EOF */
}

ssize_t sleepy_write (struct file *filp, const char __user *buf, size_t count,
                     loff_t *pos)
{
    printk(KERN_DEBUG "process %i (%s) awakening the readers...\n",
           current->pid, current->comm);
    flag = 1;
    wake_up_interruptible(&wq);
    return count; /* succeed, to avoid retrial */
}
```

- Driver `init` and `exit` functions
 - Loading and unloading drivers with `insmod/rmmod`.
- Registering character devices with `register_chrdev()`.
 - Understand difference between **major** and **minor** numbers.
- Implementing file operation functions (read, write, open, release, ioctl, etc.).
- Creating character device files.
- Maintaining data for different devices in the same driver.
 - Setting each process' file struct: operations and private data.
- *Scull* is used as a driver example, you will not need to memorize *scull*-specific implementations or data structures for the final exam.

- **Time and Place:**
 - Wednesday **5/8 7:00PM**.
 - ECEB 1002, 1013, 1015.
 - You are allowed three sheet of notes.
- **Exam Content:**
 - *Cumulative* but main focus will be on **Lectures 17-26**.
 - You still need basic concepts and knowledge from earlier lectures (*e.g., x86 assembly*).
 - Reference sheets will be provided if applicable (*e.g., driver function prototypes*).
 - MPs: **MP3** (*All with focus on Checkpoint 3-5 content*).
 - Will release more past semester final exams with solutions.



The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN